

Exercises

Exercise 6.1 Give three examples from the RISC-V architecture of each of the architecture design principles: (1) regularity supports simplicity; (2) make the common case fast; (3) smaller is faster; and (4) good design demands good compromises. Explain how each of your examples exhibits the design principle.

Exercise 6.2 The RISC-V architecture has a register set that consists of 32 32-bit registers. Is it possible to design a computer architecture without a register set? If so, briefly describe the architecture, including the instruction set. What are advantages and disadvantages of this architecture over the RISC-V architecture?

Exercise 6.3 Write the following strings using ASCII encoding. Write your final answers in hexadecimal.

- (a) hello there
- (b) bag o' chips
- (c) To the rescue!

Exercise 6.4 Repeat Exercise 6.3 for the following strings.

- (a) Cool
- (b) RISC-V
- (c) boo!

Exercise 6.5 Show how the strings in Exercise 6.3 are stored in a byte-addressable memory starting at memory address 0x004F05BC. The first character of the string is stored at the lowest byte address (in this case, 0x004F05BC). Clearly indicate the memory address of each byte.

Exercise 6.6 Repeat Exercise 6.5 for the strings in Exercise 6.4.

Exercise 6.7 The `nor` instruction is not part of the RISC-V instruction set because the same functionality can be implemented using existing instructions. Write a short assembly code snippet that has the following functionality: `s3 = s4 NOR s5`. Use as few instructions as possible.

Exercise 6.8 The `nand` instruction is not part of the RISC-V instruction set because the same functionality can be implemented using existing instructions. Write a short assembly code snippet that has the following functionality: `s3 = s4 NAND s5`. Use as few instructions as possible.

Exercise 6.9 Convert the following high-level code snippets into RISC-V assembly language. Assume that the (signed) integer variables *g* and *h* are in registers *a0* and *a1*, respectively. Clearly comment your code.

- a) `if (g > h)`
 `g = g + 1;`
 `else`
 `h = h - 1;`
- b) `if (g <= h)`
 `g = 0;`
 `else`
 `h = 0;`

Exercise 6.10 Repeat Exercise 6.9 for the following code snippets.

- a) `if (g >= h)`
 `g = g + h;`
 `else`
 `g = g - h;`
- b) `if (g < h)`
 `h = h + 1;`
 `else`
 `h = h * 2;`

Exercise 6.11 Convert the following high-level code snippet into RISC-V assembly. Assume that the base addresses of *array1* and *array2* are held in *t1* and *t2* and that the *array2* array is initialized before it is used. Use as few instructions as possible. Clearly comment your code.

```
int i;  
int array1[100];  
int array2[100];  
...  
for (i = 0; i < 100; i = i + 1)  
    array1[i] = array2[i];
```

Exercise 6.12 Repeat Exercise 6.11 for the following high-level code snippet. Assume that the *temp* array is initialized before it is used and that *t3* holds the base address of *temp*.

```
int i;  
int temp[100];  
...  
for (i = 0; i < 100; i = i + 1)  
    temp[i] = temp[i] * 128;
```

Exercise 6.13 Write RISC-V assembly code for placing the following immediates (constants) in `s7`. Use a minimum number of instructions.

- (a) 29
- (b) -214
- (c) -2999
- (d) 0xABCDE000
- (e) 0xEDCBA123
- (f) 0xEEEEEFAB

Exercise 6.14 Repeat Exercise 6.13 for the following immediates.

- (a) 47
- (b) -349
- (c) 5328
- (d) 0xBBCCD000
- (e) 0xFEEBC789
- (f) 0xCCAAB9AB

Exercise 6.15 Write a function in a high-level language for `int find42(int array[], int size)`. `size` specifies the number of elements in `array`, and `array[]` specifies the base address of the array. The function should return the index number of the first array entry that holds the value 42. If no array entry is 42, it should return the value -1. Clearly comment your code.

Exercise 6.16 The high-level function `strcpy` (string copy) copies the character string `src` to the character string `dst`.

```
// C code
void strcpy(char dst[], char src[]) {
    int i = 0;
    do {
        dst[i] = src[i];
    } while (src[i++]);
}
```

- (a) Implement the `strcpy` function in RISC-V assembly code. Use `t0` for `i`.
- (b) Draw a picture of the stack before, during, and after the `strcpy` function call. Assume `sp = 0xFFC000` just before `strcpy` is called.

Exercise 6.17 Convert the high-level function from Exercise 6.15 into RISC-V assembly code. Clearly comment your code.

This simple string copy function has a serious flaw: it has no way of knowing that `dst` has enough space to receive `src`. If a malicious programmer were able to execute `strcpy` with a long string `src`, the programmer might be able to write bytes all over memory, possibly even modifying code stored in subsequent memory locations. With some cleverness, the modified code might take over the machine. This is called a *buffer overflow attack*; it is employed by several nasty programs, including the infamous Blaster worm, which caused an estimated \$525 million in damages in 2003.

Exercise 6.18 Consider the RISC-V assembly code below. `func1`, `func2`, and `func3` are nonleaf functions. `func4` is a leaf function. The code is not shown for each function, but the comments indicate which registers are used within each function. You may assume that the functions do not need to save any nonpreserved registers on their stacks.

```

0x00091000 func1: ...    # func1 uses t2-t3, s4-s10
...
0x00091020    jal func2
...
0x00091100 func2: ...    # func2 uses a0-a2, s0-s5
...
0x0009117C    jal func3
...
0x00091400 func3: ...    # func3 uses t3, s7-s9
...
0x00091704    jal func4
...
0x00093008 func4: ...    # func4 uses s10-s12
...
0x00093118    jr  ra

```

- How many words are the stack frames of each function?
- Sketch the stack after `func4` is called. Clearly indicate which registers are stored where on the stack and mark each of the stack frames. Give values where possible. Assume that `sp = 0xABC124` just before `func1` is called.

Exercise 6.19 Each number in the Fibonacci series is the sum of the previous two numbers. Table 6.16 lists the first few numbers in the series, $fib(n)$.

- What is $fib(n)$ for $n = 0$ and $n = -1$?
- Write a function called `fib` in a high-level language that returns the Fibonacci number for any nonnegative value of n . Hint: You probably will want to use a loop. Clearly comment your code.
- Convert the high-level function of part (b) into RISC-V assembly code. Add comments after every line of code that explain clearly what it does. Use a simulator to test your code on $fib(9)$. (See the Preface for links to a RISC-V simulator.)

Table 6.16 Fibonacci series

n	1	2	3	4	5	6	7	8	9	10	11	...
$fib(n)$	1	1	2	3	5	8	13	21	34	55	89	...

Exercise 6.20 Consider [Code Example 6.28](#). For this exercise, assume $\text{factorial}(n)$ is called with input argument $n = 5$.

- (a) What value is in $a0$ when factorial returns to the calling function?
- (b) Suppose you replace the instructions at addresses $0x8508$ and $0x852C$ with nops. Will the program:
 - (1) enter an infinite loop but not crash;
 - (2) crash (cause the stack to grow or shrink beyond the dynamic data segment or the PC to jump to a location outside the program);
 - (3) produce an incorrect value in $a0$ when the program returns to loop (if so, what value?); or
 - (4) run correctly despite the deleted lines?
- (c) Repeat part (b) with the following instruction modifications:
 - (1) Replace the instructions at addresses $0x8504$ and $0x8528$ with nops.
 - (2) Replace the instruction at address $0x8518$ with a nop.
 - (3) Replace the instruction at address $0x8530$ with a nop.

Exercise 6.21 Ben Bitdiddle is trying to compute the function $f(a, b) = 2a + 3b$ for nonnegative b . He goes overboard in the use of function calls and recursion and produces the following high-level code for functions f and g .

```
// high-level code for functions f and g
int f(int a, int b) {
    int j;

    j = a;
    return j + a + g(b);
}

int g(int x) {
    int k;

    k = 3;
    if (x == 0) return 0;
    else return k + g(x - 1);
}
```

Ben then translates the two functions into RISC-V assembly language as follows. He also writes a function, `test`, that calls the function $f(5, 3)$.

```
# RISC-V assembly code
# f: a0 = a, a1 = b, s4 = j;
# g: a0 = x, s4 = k

0x8000 test: addi a0, zero, 5    # a = 5
0x8004      addi a1, zero, 3    # b = 3
0x8008      jal  f              # call f(5, 3)
0x800C loop: j    loop          # and loop forever
0x8010 f:     addi sp, sp, -16   # make room on stack
0x8014      sw   a0, 0xC(sp)    # save a0
0x8018      sw   a1, 0x8(sp)    # save a1
```

```

0x801C      sw    ra, 0x4(sp)      # save ra
0x8020      sw    s4, 0x0(sp)     # save s4
0x8024      addi  s4, a0, 0        # j = a
0x8028      addi  a0, a1, 0        # place b as argument for g()
0x802C      jal   g                # call g
0x8030      lw    t0, 0xC(sp)     # restore a into t0
0x8034      add   a0, a0, t0       # a0 = g(b) + a
0x8038      add   a0, a0, s4       # a0 = (g(b) + a) + j
0x803C      lw    s4, 0x0(sp)     # restore registers
0x8040      lw    ra, 0x4(sp)
0x8044      addi  sp, sp, 16
0x8048      jr    ra              # return
0x804C g:    addi  sp, sp, -8      # make room on stack
0x8050      sw    ra, 4(sp)       # save registers
0x8054      sw    s4, 0(sp)
0x8058      addi  s4, zero, 3      # k = 3
0x805C      bne   a0, zero, else   # if (x != 0), goto else
0x8060      addi  a0, zero, 0      # return 0
0x8064      j     done            # clean up and return
0x8068 else: addi  a0, a0, -1      # decrement x
0x806C      jal   g                # call g(x-1)
0x8070      add   a0, s4, a0       # return k + g(x-1)
0x8074 done: lw    s4, 0(sp)      # restore registers
0x8078      lw    ra, 4(sp)
0x807C      addi  sp, sp, 8
0x8080      jr    ra              # return

```

You will probably find it useful to make drawings of the stack similar to the one in [Figure 6.10](#) to help you answer the following questions.

- (a) If the code runs starting at `test`, what value is in `a0` when the program gets to `loop`? Does his program correctly compute $2a + 3b$?
- (b) Suppose Ben replaces the instruction at address `0x8014` with a `nop`. Will the program
 - (1) enter an infinite loop but not crash;
 - (2) crash (cause the stack to grow or shrink beyond the dynamic data segment or the PC to jump to a location outside the program);
 - (3) produce an incorrect value in `a0` when the program returns to `loop` (if so, what value?); or
 - (4) run correctly despite the deleted lines?
- (c) Repeat part (b) when the following instructions are changed. Note that labels aren't changed, only instructions.
 - (i) Instructions at `0x8014` and `0x8030` are replaced with `nops`.
 - (ii) Instructions at `0x803C` and `0x8040` are replaced with `nops`.
 - (iii) Instruction at `0x803C` is replaced with a `nop`.
 - (iv) Instruction at `0x8030` is replaced with a `nop`.
 - (v) Instructions at `0x8054` and `0x8074` are replaced with `nops`.

- (vi) Instructions at 0x8020 and 0x803C are replaced with nops.
- (vii) Instructions at 0x8050 and 0x8078 are replaced with nops.

Exercise 6.22 Convert the following RISC-V assembly code into machine language. Write the instructions in hexadecimal.

```
addi s3, s4, 28
sll  t1, t2, t3
srli s3, s1, 14
sw   s9, 16(t4)
```

Exercise 6.23 Repeat Exercise 6.22 for the following RISC-V assembly code:

```
add  s7, s8, s9
srai t0, t1, 0xC
ori  s3, s1, 0xABC
lw   s4, 0x5C(t3)
```

Exercise 6.24 Consider all instructions with an immediate field.

- (a) Which instructions from Exercise 6.22 use an immediate field in their machine code format?
- (b) What is the instruction type (I-, S-, B-, U-, or J-type) for the instructions from part (a)?
- (c) Write out the 5- to 21-bit immediates of each instruction from part (a) in hexadecimal. If the number is extended, also write them as 32-bit extended immediates. Otherwise, indicate that they are not extended.

Exercise 6.25 Repeat Exercise 6.24 for the instructions in Exercise 6.23.

Exercise 6.26 Consider the RISC-V machine code snippet below. The first instruction is listed at the top.

- (a) Convert the machine code snippet into RISC-V assembly language.
- (b) Reverse engineer a high-level program that would compile into this assembly language routine and write it. Clearly comment your code.
- (c) Explain in words what the program does. `a0` and `a1` are the inputs, and they initially contain positive numbers, `A` and `B`. At the end of the program, register `a0` holds the output (i.e., return value).

```
0x01800513
0x00300593
0x00000393
0x00058E33
0x01C54863
0x00138393
0x008E0E33
0xFF5FF06F
0x00038533
```

Exercise 6.27 Repeat Exercise 6.26 for the following machine code. `a0` and `a1` are the inputs. `a0` contains a 32-bit number and `a1` is the address of a 32-element array of characters (`char`).

```
0x01F00393
0x00755E33
0x001E7E13
0x01C580A3
0x00158593
0xFFFF38393
0xFE03D6E3
0x00008067
```

Exercise 6.28 Convert the following branch instructions into machine code. Instruction addresses are given to the left of each instruction.

- a) 0x0000A000 `beq t4, zero, Loop`
 0x0000A004 `...`
 0x0000A008 `...`
 0x0000A00C `Loop: ...`
- b) 0x00801000 `bne s5, a1, L1`
 `...` `...`
 0x0080174C `L1: ...`
- c) 0x0000C10C `Back: ...`
 `...` `...`
 0x0000D000 `blt s1, s2, Back`
- d) 0x01030AAC `bge t4, t6, L2`
 `...` `...`
 0x01031AA4 `L2: ...`
- e) 0x0BC08004 `L3: ...`
 `...` `...`
 0x0BC09000 `beq s3, s7, L3`

Exercise 6.29 Convert the following branch instructions into machine code. Instruction addresses are given to the left of each instruction.

- a) 0xAA00E124 `blt t4, s3, Loop`
 0xAA00E128 `...`
 0xAA00E12C `...`
 0xAA00E130 `Loop: ...`
- b) 0xC0901000 `bge t1, t2, L1`
 `...` `...`
 0xC090174C `L1: ...`

- c) 0x1230D10C Back: ...

 0x1230D908 bne s10, s11, Back
- d) 0xAB0C99A8 beq a0, s1, L2

 0xAB0CA0FC L2: ...
- e) 0xFFABCF04 L3: ...

 0xFFABD640 blt s1, t3, L3

Exercise 6.30 Convert the following jump instructions into machine code. Instruction addresses are given to the left of each instruction.

- a) 0x1234ABC0 j Loop

 0x123CABBC Loop: ...
- b) 0x12345678 Back: ...

 0x123B8760 jal s0, Back
- c) 0xAABBCCD0 jal L1

 0xAABDCD98 L1: ...
- d) 0x11223344 j L2

 0x1127BCDC L2: ...
- e) 0x9876543C L3: ...

 0x9886543C jal L3

Exercise 6.31 Convert the following jump instructions into machine code. Instruction addresses are given to the left of each instruction.

- a) 0x0000ABC0 jal Loop

 0x0000EEEC Loop: ...
- b) 0x0000C10C Back: ...

 0x000F1230 jal Back
- c) 0x00801000 jal s1, L1

 0x008FFFDC L1: ...

- d) 0xA1234560 j L2

 0xA131347C L2: ...
- e) 0xF0BBCCD4 L3: ...

 0xF0CBCCD4 j L3

Exercise 6.32 Consider the following RISC-V assembly language snippet. The numbers to the left of each instruction indicate the instruction address.

```
0xA0028 Func1: addi t4, a1, 0
0xA002C        ori  a0, a0, 32
0xA0030        sub  a1, a1, a0
0xA0034        jal  Func2
...
0xA0058 Func2: lw   t2, 4(a0)
0xA005C        sw   t2, 16(a1)
0xA0060        srli t3, t2, 8
0xA0064        beq  t2, t3, Else
0xA0068        jr   ra
0xA006C Else:  addi a0, a0, 4
0xA0070        j    Func2
```

- Translate the instruction sequence into machine code. Write the machine code instructions in hexadecimal.
- List the instruction type and addressing mode used at each line of code.

Exercise 6.33 Consider the following C code snippet.

```
// C code
void setArray(int num) {
    int i;
    int array[10];

    for (i = 0; i < 10; i = i + 1)
        array[i] = compare(num, i);
}

int compare(int a, int b) {
    if (sub(a, b) >= 0)
        return 1;
    else
        return 0;
}

int sub(int a, int b) {
    return a - b;
}
```

- Implement the C code snippet in RISC-V assembly language. Use `s4` to hold the variable `i`. Be sure to handle the stack pointer appropriately. The array is stored on the stack of the `setArray` function (see the end of [Section 6.3.7](#)). Clearly comment your code.

- (b) Assume that `setArray` is the first function called. Draw the status of the stack before calling `setArray` and during each function call. Indicate stack addresses and the names of registers and variables stored on the stack; mark the location of `sp`; and clearly mark each stack frame. Assume that `sp` starts at `0x8000`.
- (c) How would your code function if you failed to store `ra` on the stack?

Exercise 6.34 Consider the following high-level function.

```
// C code
int f(int n, int k) {
    int b;

    b = k + 2;
    if (n == 0)
        b = 10;
    else
        b = b + (n * n) + f(n - 1, k + 1);
    return b * k;
}
```

- (a) Translate the high-level function `f` into RISC-V assembly language. Pay particular attention to properly saving and restoring registers across function calls and using the RISC-V preserved register conventions. Clearly comment your code. Assume that the function starts at instruction address `0x8100`. Keep local variable `b` in `s4`. Clearly comment your code.
- (b) Step through your function from part (a) by hand for the case of `f(2, 4)`. Draw a picture of the stack similar to the one in [Figure 6.10](#), and assume that `sp` is equal to `0xBFF00100` when `f` is called. Write the stack addresses and the register name and data value stored at each location in the stack and keep track of the stack pointer value (`sp`). Clearly mark each stack frame. You might also find it useful to keep track of the values in `a0`, `a1`, and `s4` throughout execution. Assume that when `f` is called, `s4 = 0xABCD` and `ra = 0x8010`.
- (c) What is the final value of `a0` when `f(2, 4)` is called?

Exercise 6.35 What is the maximum number of instructions that a branch instruction (like `beq`) can branch forward (i.e., to higher instruction addresses)?

Exercise 6.36 What is the maximum number of instructions that a branch instruction (like `beq`) can branch backward (i.e., to lower instruction addresses)?

Exercise 6.37 Write assembly code that conditionally branches to an instruction 32 Minstructions forward from a given instruction. Recall that 1 Minstruction = 2^{20} instructions = 1,048,576 instructions. Assume that your code begins at address `0x8000`. Use a minimum number of instructions.

Exercise 6.38 Explain why it is advantageous to have a large immediate field in the machine format for the jump and link instruction, `jal`.

Exercise 6.39 Consider a function that takes a 10-entry array of 32-bit integers stored in little-endian format and converts it to big-endian format.

- (a) Write this function in high-level code.
- (b) Convert this function to RISC-V assembly code. Comment all your code and use a minimum number of instructions.

Exercise 6.40 Consider two strings: `string1` and `string2`.

- (a) Write high-level code for a function called `concat` that concatenates (joins together) the two strings: `void concat(char string1[], char string2[], char stringconcat[]`). The function does not return a value. It concatenates `string1` and `string2` and places the resulting string in `stringconcat`. You may assume that the character array `stringconcat` is large enough to accommodate the concatenated string. Clearly comment your code.
- (b) Convert the function from part (a) into RISC-V assembly language. Clearly comment your code.

Exercise 6.41 Write a RISC-V assembly program that adds two positive single-precision floating-point numbers held in `a0` and `a1`. Do not use any of the RISC-V floating-point instructions. You need not worry about any of the encodings that are reserved for special purposes (e.g., 0, NaNs, etc.) or numbers that overflow or underflow. Use a simulator to test your code. (See the Preface for links to a RISC-V simulator.) You will need to manually set the values of `a0` and `a1` to test your code. Demonstrate that your code functions reliably. Clearly comment your code.

Exercise 6.42 Expand the RISC-V assembly program from Exercise 6.41 to handle both positive and negative single-precision floating-point numbers. Clearly comment your code.

Exercise 6.43 Consider a function that sorts a 10-element array called `scores` from lowest to highest. After the function completes, `scores[0]` holds the smallest value and `scores[9]` holds the highest value.

- (a) Write a high-level `sort` function that performs the function above. `sort` receives a single argument, the address of the `scores` array. Clearly comment your code.
- (b) Convert the `sort` function into RISC-V assembly language. Clearly comment your code.

Exercise 6.44 Consider the following RISC-V program. Assume that the instructions are placed starting at memory address 0x8400 and that global variables *x* and *y* are at memory addresses 0x10024 and 0x10028, respectively.

```
# RISC-V assembly code
main:
    addi sp, sp, -4    # make room on stack
    sw   ra, 0(sp)     # save ra on stack
    lw   a0, -940(gp)  # a0 = x
    lw   a1, -936(gp)  # a1 = y
    jal  diff          # call diff()
    lw   ra, 0(sp)     # restore registers
    addi sp, sp, 4
    jr   ra            # return
diff:
    sub  a0, a0, a1    # return (a0-a1)
    jr   ra
```

- First, show the instruction address next to each assembly instruction.
- Describe the symbol table: that is, list the name, address, and size of each symbol (i.e., function label and global variable).
- Convert all instructions into machine code.
- How big (how many bytes) are the data and text segments?
- Sketch a memory map showing where data and instructions are stored, similar to [Figure 6.34](#). Be sure to label the values of PC and gp at the beginning of the program.

Exercise 6.45 Repeat Exercise 6.44 for the following RISC-V code. Assume that the instructions are placed starting at memory address 0x8534 and that global variables *g* and *h* are at memory addresses 0x1305C and 0x13060.

```
# RISC-V assembly code
main:
    addi sp, sp, -8
    sw   ra, 4(sp)
    sw   s4, 0(sp)
    addi s4, zero, 15
    sw   s4, -300(gp)  # g = 15
    addi a1, zero, 27  # arg1 = 27
    sw   a1, -296(gp)  # h = 27
    lw   a0, -300(gp)  # arg0 = g = 15
    jal  greater
    lw   s4, 0(sp)
    lw   ra, 4(sp)
    addi sp, sp, 8
    jr   ra
greater:
    blt  a1, a0, isGreater
    addi a0, zero, 0
    jr   ra
isGreater:
    addi a0, zero, 1
    jr   ra
```

Exercise 6.46 Explain the advantages and disadvantages of the bit-swizzling found in the encodings of RISC-V immediates.

Exercise 6.47 Consider sign-extension of the immediates in RISC-V instructions. Design a sign-extension unit for RISC-V immediates using the steps below. Minimize the amount of hardware needed.

- Sketch the schematic for a sign-extension unit that sign-extends the 12-bit immediate in I-type instructions. The circuit's input is the upper 12 bits of the instruction, $Instr_{31:20}$, which encodes the 12-bit signed immediate. The output is a 32-bit sign-extended immediate, $ImmExt_{31:0}$.
- Expand the sign-extension unit from part (a) to also sign-extend the 12-bit immediate found in S-type instructions. Modify the inputs as needed, and reuse hardware when possible.
- Expand the sign-extension unit from part (b) to also sign-extend the 13-bit immediate found in B-type instructions.
- Expand the sign-extension unit from part (c) to also sign-extend the 21-bit immediate found in J-type instructions.

Exercise 6.48 In this exercise, you will design an alternate extension unit for RISC-V immediates using a minimum amount of hardware. Suppose that the RISC-V architects had chosen to use immediate encodings that were more straightforward to humans, as shown in Figure 6.38. This figure shows all instruction fields except op. The immediate encodings in Figure 6.38 use no bit-swizzling (but still split the immediate field in S/B-type instructions across two instruction fields). Bits that differ from the actual RISC-V immediate encodings (shown in Figure 6.27) are highlighted in blue. Specifically, these hypothetical (straightforward) immediate encodings differ from actual RISC-V immediate encodings for the B- and J-type formats.

- Sketch the schematic for a sign-extension unit that sign-extends the 12-bit immediate in I-type instructions. The circuit's input is the upper 12 bits of

11	10	9	8	7	6	5	4	3	2	1	0	rs1	funct3	rd	I										
11	10	9	8	7	6	5	rs2					rs1	funct3	4	3	2	1	0	S						
12	11	10	9	8	7	6	rs2					rs1	funct3	5	4	3	2	1	B						
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	rd	U				
20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	rd	J				
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	

Figure 6.38 Alternate immediate encodings

the instruction, $Instr_{31:20}$, which encodes the 12-bit signed immediate. The output is a 32-bit sign-extended immediate, $ImmExt_{31:0}$.

- (b) Expand the sign-extension unit from part (a) to also sign-extend the 12-bit immediate found in S-type instructions. Modify the inputs as needed, and reuse hardware when possible.
- (c) Expand the sign-extension unit from part (b) to also sign-extend the 13-bit immediate found in (the modified) B-type instructions (see Figure 6.38).
- (d) Expand the sign-extension unit from part (c) to also sign-extend the 21-bit immediate found in (the modified) J-type instructions (see Figure 6.38).
- (e) If you completed Exercise 6.47, compare this to the hardware in the actual RISC-V extension unit.

Exercise 6.49 Consider how far `jal` instructions can jump.

- (a) How many instructions can a `jal` instruction jump forward (i.e., to higher addresses)?
- (b) How many instructions can a `jal` instruction jump backward (i.e., to lower addresses)?

Exercise 6.50 Consider memory storage of a 32-bit word stored at the 42nd memory word in a byte-addressable memory. Remember, the 0th word is stored at memory address 0, the first word at address 4, and so on.

- (a) What is the byte address of the 42nd word stored in memory?
- (b) What are the byte addresses that the 42nd word spans?
- (c) Draw the number 0xFF223344 stored at word 42 in both big-endian and little-endian machines. Clearly label the byte address corresponding to each data byte value.

Exercise 6.51 Repeat Exercise 6.50 for memory storage of a 32-bit word stored at the 15th word in a byte-addressable memory.

Exercise 6.52 Explain how the following RISC-V program can be used to determine whether a computer is big-endian or little-endian:

```
addi s7, 100
lui  s3, 0xABCD8      # s3 = 0xABCD8000
addi s3, s3, 0x765    # s3 = 0xABCD8765
sw   s3, 0(s7)
lb   s2, 1(s7)
```